

Structural Damage Image Classification

ML Mini Project | Two-class image classification of undamaged and damaged structures

1. Problem Statement

Structural inspection produces large collections of images that must be screened for visible damage. Manual inspection is valuable but time-consuming and can be inconsistent at scale. This project studies whether machine-learning models can classify an image as undamaged or damaged, where the damaged class includes cracks and other structural damage. The implementation reproduces the model families reported in *Structural Damage Image Classification* by Ho and Troncoso (2018).

The objective is not to replace an engineer's inspection. It is to build and compare practical image classification baselines, identify useful feature representations, and provide a reproducible workflow for a damage-screening experiment.

2. Dataset

The project uses the PEER Hub ImageNet Challenge Task 2 Damage State dataset. The supplied NumPy arrays contain 11,811 training images and 1,460 test images. Each image has shape 224 x 224 x 3. The source arrays use Caffe-style, mean-subtracted BGR values. The loader restores the channel means, reverses the channels to RGB, and normalizes each pixel with $x = (x / 128) - 1$.

Item	Project detail
Training data	11,811 images and one-hot labels
Held-out data	1,460 images and one-hot labels
Classes	0 = undamaged; 1 = damaged
Split	Stratified 90% train / 10% validation, seed 42

Because the labels contain only damaged versus undamaged states, the project cannot report a separate crack-only accuracy. Cracks are evaluated as part of the damaged class.

3. Approach

The workflow compares three classical models with three deep-feature models. Classical models receive flattened 224 x 224 x 3 images (150,528 features) standardized using a scaler fitted only on the training split. The transfer-learning models use ImageNet pretrained networks with frozen convolutional bases and a two-class classification head. The hybrid model uses intermediate InceptionV3 activations as input to an RBF-kernel SVM.

4. Implementation

- **KNN**: $k = 5$ with brute-force distance search.
- **Logistic regression**: L2 regularization, $C = 1.0$, LBFGS solver.
- **SVM**: RBF kernel with $C = 1.0$ and $\gamma = 0.001$.
- **MobileNetV1**: frozen ImageNet base, global average pooling, and a two-class head.
- **InceptionV3**: frozen ImageNet base with SGD learning rate 0.01.
- **Hybrid**: pooled layer-288 InceptionV3 features, standardization, and γ sweep from $1e-6$ to $1e-1$.

5. Experimental Results

To provide a safe, reproducible proof run on a local Windows system, all six models were executed sequentially with one process at a time. Smoke mode selected a stratified sample of 150 images, giving 135 training images and 15 validation images. TensorFlow thread counts were limited to one. Deep models ran for two epochs. The following results are from the recorded run on 2026-10-07.

Model	Train	Validation	Time
KNN (k=5)	68.89%	66.67%	0.99 s
Logistic regression	100.00%	53.33%	2.77 s
SVM RBF	100.00%	53.33%	4.30 s
MobileNetV1	73.33%	66.67%	12.96 s
InceptionV3	58.52%	53.33%	31.79 s
InceptionV3 layer 288 + SVM	87.41%	73.33%	13.89 s

The strongest validation result in this capped run was the InceptionV3 layer-288 hybrid at 73.33%, followed by MobileNetV1 at 66.67% and KNN at 66.67%. Logistic regression and the raw-pixel RBF SVM both reached 100% training accuracy but only 53.33% validation accuracy, indicating overfitting on the small smoke sample. The hybrid gamma sweep selected gamma = 0.001.

6. Conclusion

The experiments demonstrate a complete damage-classification pipeline from raw NumPy image arrays to classical, transfer-learning, and hybrid predictions. Learned visual features were more useful than the flattened pixel baselines in this smoke evaluation, with the Inception-SVM representation providing the best validation accuracy. The result is promising for automated screening, but the 15-image validation set is too small for a final performance claim.

A stronger final study should run the full training split, evaluate the held-out 1,460-image test set, report precision, recall, F1-score, and a confusion matrix, and use fixed random seeds for deep-learning reproducibility. Any operational deployment would also require domain expert review and images from conditions representative of the intended inspection environment.

Implementation: `code/common.py` and `code/01_knn.py` through `code/06_inception_svm_hybrid.py`. Proof runner: `run_smoke_150.py`. Results: `results/results_summary.csv` and `run_logs/`.